



分支限界法

《算法分析与设计》

罗建超

长聘副教授、博士生导师

目录

1

- 分支限界法基本思想

2

- 分支定界法设计要素

3

- 案例分析：装载问题

4

- 其它案例分析

分支限界法基本思想

分支界限法与回溯法类似，也是在问题解空间中搜索问题解的一种算法。

分支界限法与回溯法对比：

求解目标不同：回溯法的可以用于求解目标是找出解空间树中满足约束条件的**所有解**，而分支限界法的求解目标**通常是**找出满足约束条件的**一个解或最优解**。

搜索方式不同：回溯法**主要**以**深度优先**的方式搜索解空间树，而分支限界法则**主要**以**广度优先或以函数优先**的方式搜索解空间树。

分支限界法基本思想

■ 在分支限界法中，每个活结点只有一次机会成为扩展结点。一旦成为扩展结点，就一次性产生其所有儿子结点。在这些儿子结点中，导致不可行解或导致非最优解的儿子结点被舍弃，其余儿子结点被加入活结点表中。

■ 此后，从活结点表中取下一结点成为当前扩展结点，并重复上述结点扩展过程。这个过程一直持续到找到所需的解或活结点表为空时为止。

分支限界法基本思想—代码框架

$Q = \{q_0\}$; // 存储所有的活结点, 初始化为根结点

void Branch&Bound ()

{

while ($Q \neq \emptyset$) {

select a node q from Q ; // 从 Q 选择一个结点

Branch(q, Q_1); // 对 q 进行分支, 产生 Q_1 , 分支
时利用约束和界进行剪枝

add (Q_1, Q); // 将新产生的活结点加入 Q

}

}

分支限界法基本思想

常见的两种分支搜索法（选择结点方式）

队列式(FIFO)搜索法：按照队列先进先出(FIFO)原则选取下一个结点为扩展结点。

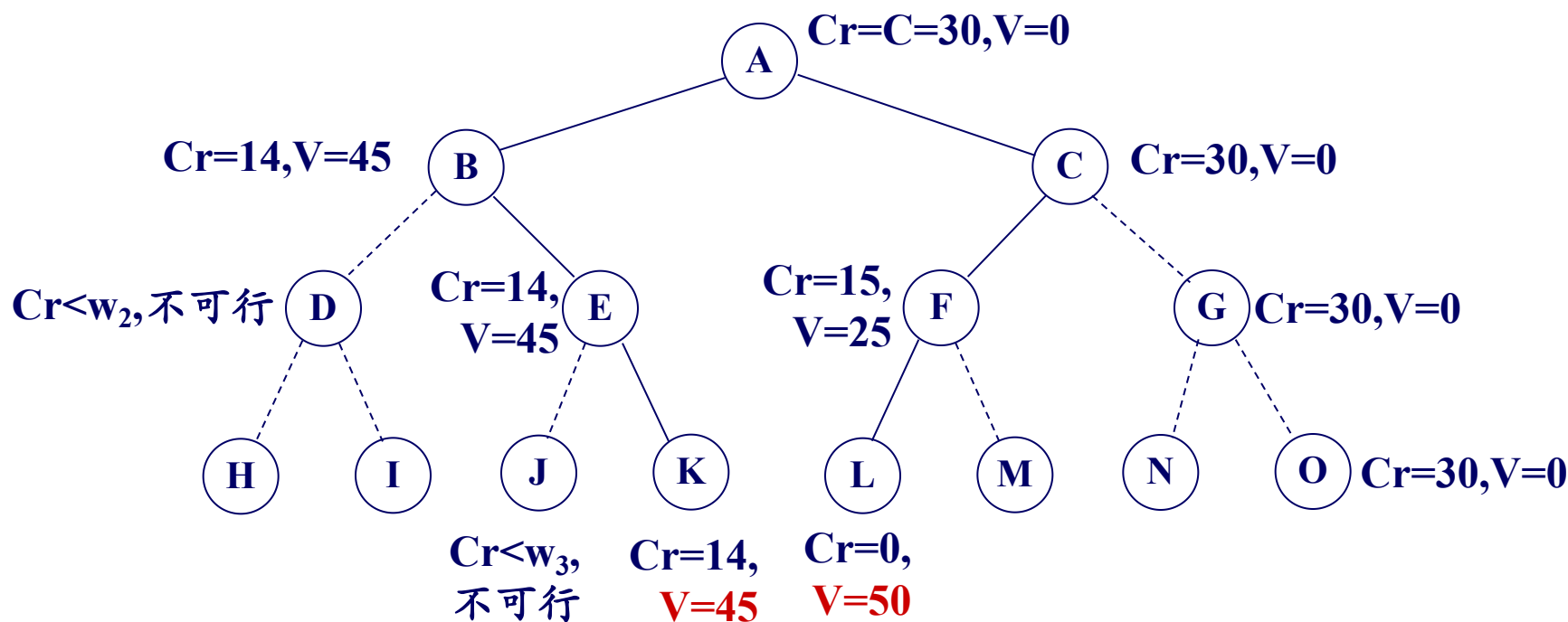
优先队列式搜索法：按照优先队列中规定的优先级选取优先级最高的结点成为当前扩展结点。

- 最大堆（最大优先队列）：最大效益优先
- 最小堆（最小优先队列）：最小耗费优先

分支限界法基本思想

0-1 背包问题：（分析队列式与优先队列式过程）

$n=3$, $C=30$, $w=\{16, 15, 15\}$, $v=\{45, 25, 25\}$



A	B	C	E	F	K	L
---	---	---	---	---	---	---

A	B	C	E	K	F	L
---	---	---	---	---	---	---

分支限界法基本思想—上界&下界

结点 v 上界($UB(v)$): 从 v 出发得到的所有叶子结点的效益值均**不大于** $UB(v)$, 则 $UB(v)$ 为结点 v 的上界;

如果所有叶子结点的最大效益值**等于** $UB(v)$, 则 $UB(v)$ 为结点 v 的**上确界**;

上界越大越好, 还是越小越好?

分支限界法基本思想—上界&下界

结点 v 下界($LB(v)$): 从 v 出发得到的所有叶子结点的效益值均**不小于** $LB(v)$, 则 $LB(v)$ 为结点 v 的下界;

如果所有叶子结点的最小效益值**等于** $LB(v)$, 则 $LB(v)$ 为结点 v 的**下确界**。

下界越大越好, 还是越小越好?

分支限界法基本思想—极小化

◆对于求最小值的优化问题，如果 $LB(v) \geq cBest$ ，则结点 v 可以加入黑名单，不再对其搜索。

◆ $UB(v)$ 通常可以利用贪心思路或者其它方式得到一个解，令其作为 $UB(v)$ ，而 $LB(v)$ 通常需要经过严格的证明。

分支限界法基本思想—极小化

- ◆ 对于求最小值的优化问题，如果 $UB(v_0) = LB(v_0)$ ，则直接结束，输出对应于 $UB(v_0)$ 的解即可；
- ◆ 如果结点 v 的 $UB(v) = LB(v)$ ，则该结点不用继续搜索，直接用 $UB(v)$ 作为其可到达的最优值，对 $cBest$ 进行更新；
- ◆ 如果找到叶结点 v 使得 $f(v) = LB(v_0)$ （ $f(v)$ 是 v 的真实目标值）或者所有的结点搜索完毕 则搜索完毕。
- ◆ $cBest$ 初始等于 $UB(v_0)$ ，找到 $UB(v)$ 小于 $cBest$ ，则可以更新 $cBest$ ，另外可以剪掉 $LB(v) \geq cBest$ 的点 v 。

极小化问题主要用上界还是下界剪枝？

分支限界法基本思想—极大化

◆ 对于求最大值的优化问题，如果 $UB(v) \leq cBest$ ，则结点 v 可以加入黑名单，不再对其搜索。

◆ $LB(v)$ 通常可以利用贪心思路或者其它方式得到一个解，令其作为 $LB(v)$ ，而 $UB(v)$ 通常需要经过严格的证明。

分支限界法基本思想—极大化

- ◆如果 $UB(v_0) = LB(v_0)$ ，则直接结束，输出对应于 $LB(v_0)$ 的解即可；
- ◆如果结点 v 的 $UB(v) = LB(v)$ ，则该结点不用继续搜索，直接用 $LB(v)$ 作为其可到达的最优值，对 $cBest$ 进行更新；
- ◆如果找到叶结点 v 使得 $f(v) = UB(v_0)$ 或者所有的结点搜索完毕 则搜索完毕。
- ◆ $cBest$ 初始等于 $LB(v_0)$ ，找到 $LB(v)$ 大于 $cBest$ ，则可以更新 $cBest$ ，另外可以剪掉 $UB(v) \leq cBest$ 的点 v 。

极大化问题主要用上界还是下界剪枝？

分支限界法基本思想—上界&下界

0-1背包问题：（极大值问题）

$n=3, C=30, w=\{16, 15, 15\}, v=\{28, 30, 30\}$

物品按照价值比重量的大小进行排序： $\{w_2, w_3, w_1\}$ 。

下界：按照价值比重量的大小依次装包，在满足容量约束的前提下得到的最大值作为下界LB，即 $LB(v_0) = 30 + 30 = 60$ ，对应的解为 $(0, 1, 1)$

上界：按照价值比重量的大小依次装包，没装满的按照未装的物品的最大比重进行换算，即 $UB(v_0) = 30 + 30 = 60$ 。

$LB(v_0) = UB(v_0)$ ，故最优解为 $(0, 1, 1)$ 。

分支限界法基本思想—上界&下界

0-1背包问题:

$n=3, C=30, w=\{16, 15, 15\}, v=\{45, 25, 25\}$

物品按照价值比重量的大小进行排序: $\{w_1, w_2, w_3\}$.

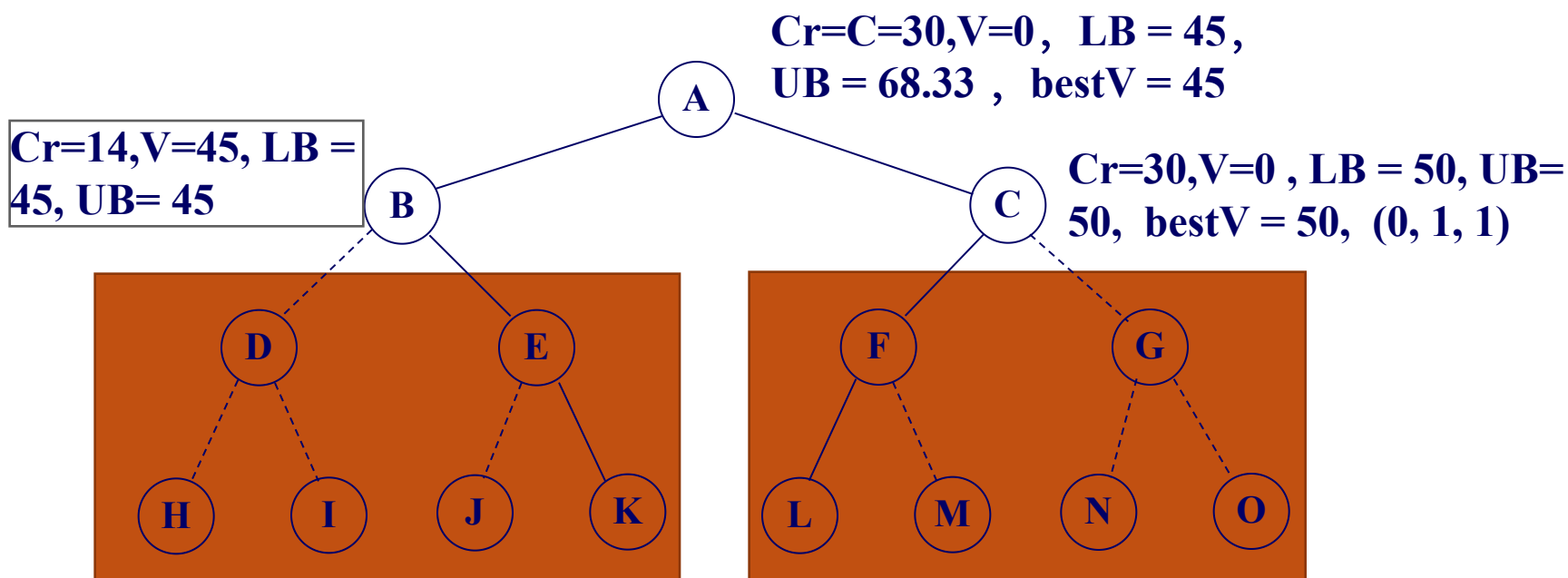
如果 $n=4, C=30, w=\{40, 16, 15, 15\}, v=\{160, 45, 25, 25\}$, 不去掉物品1, 上界为 $UB_1(v_0) = 30 \times 160 / 40 = 120 \gg 68.33$ 。

上界: 先把物品重量大于余量的除掉, 剩下物品按照价值比重量的大小依次装包, 最后没装满的按照未装的物品的最大比重进行换算, 即 $UB_1(v_0) = 45 + 14 \times 25 / 15 = 68.33$ 。

分支限界法基本思想

0-1 背包问题:

$n=3$, $C=30$, $w=\{16, 15, 15\}$, $v=\{45, 25, 25\}$



目录

1

- 分支限界法基本思想

2

- 分支定界法设计要素

3

- 案例分析：装载问题

4

- 其它案例分析

分支限界法设计要素

◆ 针对问题定义解空间：

- 问题解向量

- 解向量分量取值集合

- 构造解空间树

◆ 判断问题是否满足多米诺性质

◆ 确定剪枝函数与界函数

分支限界法设计要素

- ◆**适用条件**：与回溯法相同，问题需要具备**多米诺**性质。
- ◆**适用问题**：适合于求解**最优解**或一个**可行解**
- ◆**程序结构**：结点扩展适合采用**迭代法**进行。

目录

1

- 分支限界法基本思想

2

- 分支定界法设计要素

3

- 案例分析：装载问题

4

- 其它案例分析

分支限界法—装载问题

有一批共 n 个集装箱要装上2艘载重量分别为 c_1 和 c_2 的轮船，其中集装箱 i 的重量为 w_i ，且 $\sum_{i=1}^n w_i \leq c_1 + c_2$ 。

装载问题要求确定是否有一个合理的装载方案可将这批集装箱装上这2艘轮船。如果有，找出一种装载方案。

例如：当 $n = 3, c_1 = c_2 = 50$ ，且 $w = [10, 40, 40]$ 时，可将1和2装上第一艘轮船，3装入第二艘轮船；若 $w = [20, 40, 40]$ ，则无法将这三个集装箱都装上轮船。

分支限界法—装载问题

容易证明，如果一个给定装载问题有解，则采用下面的策略可得到一个装载方案。

(1) 首先将第一艘轮船尽可能装满；

(2) 将剩余的集装箱装上第二艘轮船。

将第一艘轮船尽可能装满等价于选取全体集装箱的一个子集，使该子集中集装箱重量之和最接近。由此可知，装载问题等价于以下特殊的0-1背包问题。

问题模型

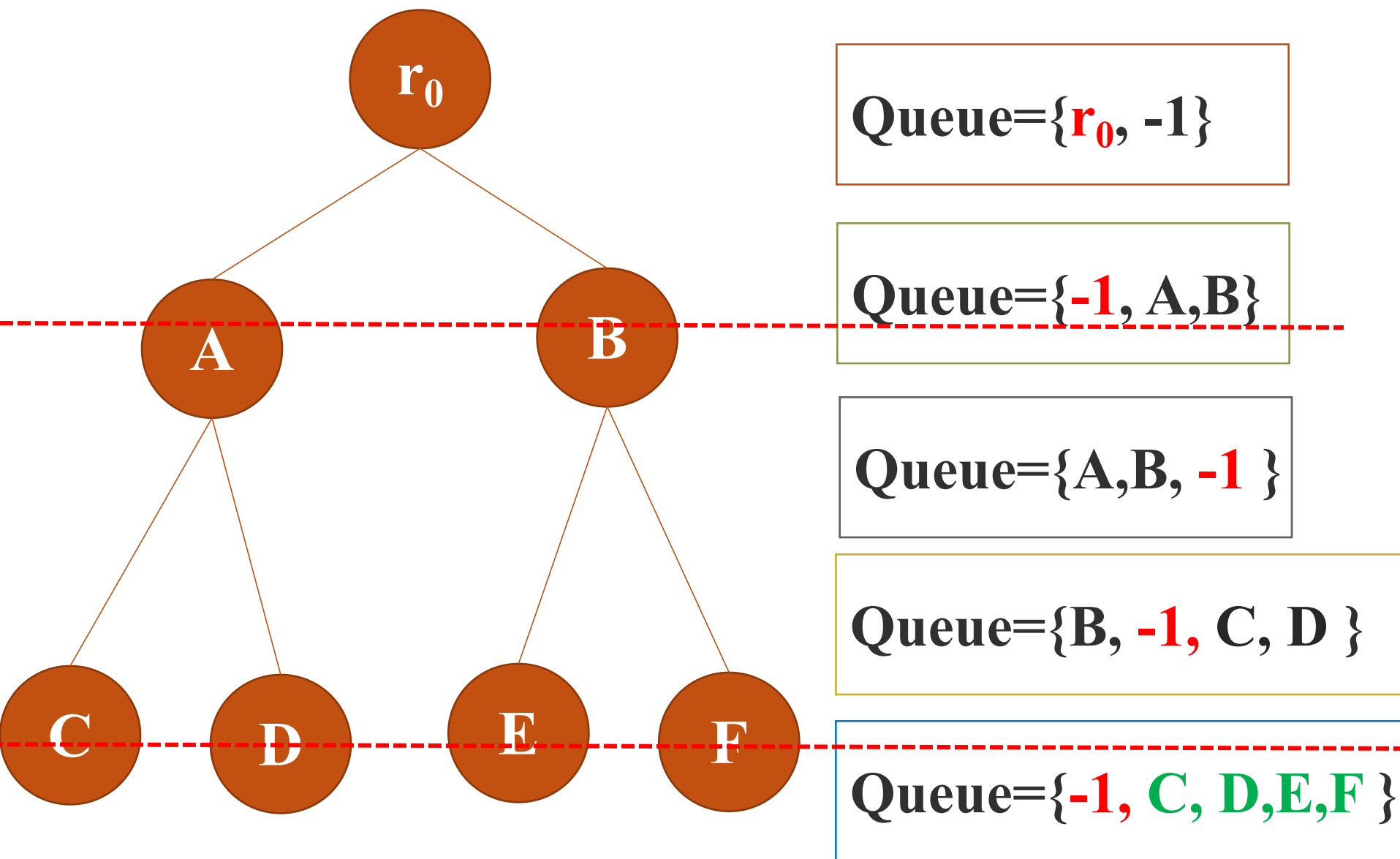
$$\begin{aligned} & \max \sum_{i=1}^n w_i x_i \\ & \text{s.t.} \sum_{i=1}^n w_i x_i \leq c_1 \\ & x_i \in \{0,1\}, 1 \leq i \leq n \end{aligned}$$

分支限界法—装载问题(FIFO)

算法思想：问题的解空间为子集树，搜索空间为 2^n ， n 为物品的数量。

- 1) 用FIFO分支搜索所有的分支，并记录已搜索的分支的最优解，搜索完子集树，就找到了问题的解。
- 2) 要想求出最优解，必须搜索到**叶结点**。需要记录树的层次，**可以每处理完一层让-1入队，来识别层数。**

分支限界法—装载问题(FIFO)



分支限界法—装载问题(FIFO)

算法设计:

每个活结点要记录哪些信息?

- 1、当前载重量;
- 2、当前层数;
- 3、当前物品的选择情况。

- 1、必须，否则需要重复计算;
- 2、当前层数，可以统一记录，避免重复;
- 3、如果只求最大载重则不需要记录，如果需要给出方案则需要记录，物品选择情况可以存储树优化，不必每个点都存储。

装载问题(FIFO)

```
int MaxLoadingFIFO (int w[], int c, int n) { // 队列式分支限界法, 返回最优载重
    Q.add(-1); // Q为活结点队列, 加入队列标记分层
    int i = 1; // 根结点结点的层
    cw = 0; // 根结点当前的载重
    bestw = 0; // 目前的最优值
    while( !Q.isEmpty()) {
        if (cw + w[i] <= c) AddLiveNode(cw+w[i], i+1); // 添加左儿子结点
        AddLiveNode(cw, i+1); // 添加右儿子结点
        cw=Q.poll(); // 从Q中取下一个扩展结点, 并赋给cw
        if (cw == -1) { // 同层尾部结点
            if(Q.isEmpty()) return bestw;
            Q.add(-1); // 同层尾部标识
            cw=Q.poll(); // 取下一个扩展结点
            i++; // 进入下一层
        }
    }
}
```

```
void AddLiveNode (int cw, int i) {
    if (i == (n+1))
        if(cw > bestw) bestw = cw;
    else
        Q.add(cw) }
```

分支限界法—装载问题(FIFO 改进)

存在问题：

- 没有给出具体的装载方案；
- 没有加入限界的技巧，属于盲目搜索；
- 在深度为 $n+1$ 的时候才更新最优解，实际上在 $n+1$ 之前的任何一个深度都可以更新最优解；

分支限界法—装载问题(FIFO 改进)

解决办法：

➤ **问题一**：存储二叉树，每个结点都能**指向父结点的指针**，方便从叶子结点回溯找到解，另外还需要记录当前结点是父结点的**哪一个儿子**；

➤ **问题二**：令 $UB = cw + r$ ，如果 $UB \leq bestw$ 时可将结点剪去；

➤ **问题三**：如果 $cw > bestw$ 即可对 $bestw$ 进行更新，**不需要等到叶子结点**。

分支限界法—装载问题(FIFO 改进)

数据结构：二叉树，树的结点的信息包括**已装船**的集装箱的重量cw，父指针（java存父结点），parent，**是否左儿子Lchild**；

Qnode {

int cw;

Qnode parent;

boolean Lchild;

}

装载问题(FIFO 改进)

```
Queue<Qnode> Q = new LinkedList<Qnode>();
Qnode bestNode;
int bestw = 0; // 目前的最优值
Vector<Boolean> bestH = new Vector<Boolean>();
void MaxLoadingFIFOImpr (int w[], int c, int n) { // 队列式
    Q.add(new Qnode(-1, null, false)); // 初始化结点队列标记分层
    int i = 1; // 扩展结点的层
    int cw = 0; // 扩展结点当前的载重
    int r = 0; // 记录剩余容量
    for(int j = 2; j <= n; j++)
        r = r + w[j];
    Qnode cQnode = null; // 根结点
```


装载问题(FIFO 改进)

```
while( !Q.isEmpty()) {  
    if (cw + w[i] <= c) {  
        //左子树判断约束  
        if(cw + w[i] > bestw) {  
            //更新bestw  
            bestw = cw + w[i];  
            //提前更新bestW  
            bestNode = new Qnode(bestw, cQnode, true);  
            AddLiveNode(cw+w[i], cQnode, true, i+1);  
            //添加左儿子结点  
        }  
        if(cw + r > bestw) {  
            //利用上界进行剪枝  
            AddLiveNode(cw, cQnode, false, i+1);  
            //添加右儿子结点  
        }  
        cQnode = Q.poll();  
        //取下一个扩展结点, 并赋值给cQnode  
        cw = cQnode.cw;  
        if (cQnode.cw == -1) {  
            //同层尾部结点  
            if(Q.isEmpty()) break;  
            Q.add(new Qnode(-1, null, false));  
            //同层尾部标识  
            cQnode = Q.poll();  
            //取下一个扩展结点  
            cw = cQnode.cw;  
            i++;  
            //进入下一层  
            r = r - w[i];  
        }  
    }  
}
```

装载问题(FIFO 改进)

```
while (bestNode != null) { //查找记录最优解
    bestH.add(bestNode.Lchild);
    bestNode = bestNode.parent;
}
}
```

```
void AddLiveNode (int cw, Qnode parent, bool Lchild, int i) {
    if (i == (n+1))
        if(cw > bestw) bestw = cw;
    else
        Q.add(cw, parent, Lchild)
}
```

分支限界法—装载问题(PriorQueue)

存在问题：

加入了界，但是搜索还是属于盲目搜索。先搜索优先级高的结点能够尽快朝较好的方向分支，可以较快得到一个较好的解，有利于后面搜索过程的剪枝，从而减少搜索量。

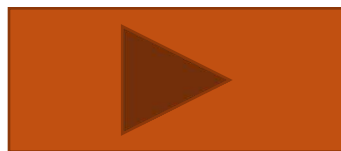
解决办法：

- 将活结点组织成一个优先队列，并按优先队列中规定的结点的优先级对结点进行扩展。
- 结点的优先级可以通过结点的上界来定；

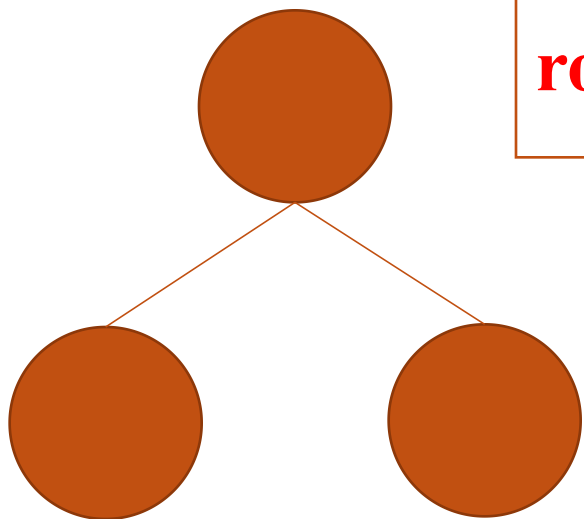
分支限界法—装载问题(PriorQueue)

数据结构设计:

- 要输出解的方案，搜索过程中仍需要生成解空间树，结点信息包括父结点、Lchild, uWeight(优先级信息)。
- 优先队列无法体现层 (level) 信息，只能存储在结点中。
- 需要用r[]来记录各层的最大剩余重量，例如 $r[1] = w[1] + w[2] + w[3] + \dots + w[n]$
- 结点的优先级信息用 $uWeight = UB(v) = cw + r[level]$;



分支限界法—装载问题(PriorQueue)



root结点 level = 1

root儿子点
level = 2

优点: **level**=1
的时候, 确定
 $x[\text{level}]$ 的值,
只有知道**level**,
才知道确定哪
个物品

$$r[1] = w[1] + w[2] + \dots + w[n]$$

$$r[2] = w[2] + \dots + w[n]$$

$$r[n-1] = w[n-1] + w[n]$$

$$r[n] = w[n]$$

优点: **level**=2时, 计算
剩余可选物品重量和即
为 $w[2] + w[3] + \dots + w[n] =$
 $r[2] = r[\text{level}]$

分支限界法—装载问题(PriorQueue)

数据结构:

Node {

int uweight; //载重上界 (包含了载重信息)

int level; //所在层

Node parent; //父结点

boolean leftCh; //是否为左

}

分支限界法—装载问题(PriorQueue)

数据结构:

```
Comparator<Node> cmp = new Comparator<Node>()  
{  
    public int compare(Node e1, Node e2) {//从大到小  
        return e2.uweight - e1.uweight;  
    }  
};  
  
PriorityQueue<Node> heap = new PriorityQueue  
<Node> (100, cmp)
```

分支限界法—装载问题(PriorQueue)

```
int bestw = 0;
Vector<Boolean> bestH = new Vector<Boolean>(); // 记录最优解
Node bestNode;

public void MaxLoadingPrior(int[] w, int c, int n) {
    Node cNode = null; // 当前扩展结点
    int i = 1; // 扩展结点的层
    int cw = 0; // 扩展结点当前的载重
    int[] r = new int[n+2]; // 定义剩余重量数组
    r[n+1] = 0;
    for(int j = n; j > 0; j--)
        r[j] = r[j+1] + w[j];
```


分支限界法—装载问题(PriorQueue)

```
while(i != (n+1)) //如果搜索到叶子结点层，且该结点的上界所有结点的上界中
是最大的，那么从其它结点出发到达的叶子结点的最大装载量都不可能比这个值大{
    if(cw + w[i] <= c)    { //搜索左子树
        if(cw + w[i] > bestw) //更新bestw { 提前更新bestW
            bestw = cw + w[i]; //todo 清理heap中上界小于bestw的结点
            bestNode = new Node(cw+w[i]+r[i+1], i+1, cNode, true); }
            AddLiveNode(cw+w[i]+r[i+1], i+1, cNode, true); }
        if(cw + r[i+1] > bestw) //利用上界进行剪枝
            AddLiveNode(cw+r[i+1], i+1, cNode, false);
        if(heap.size() == 0) break; //取下一个结点
        cNode = heap.poll();
        i = cNode.level;
        cw = cNode.uweight-r[i];
    }
    while(bestNode!=null) { //保留最优解
        bestH.add(bestNode.leftCh);
        bestNode = bestNode.parent; }
}
```

利用上界进行剪枝

目录

1

- 分支限界法基本思想

2

- 分支定界法设计要素

3

- 案例分析：装载问题

4

- 其它案例分析

0-1 背包问题

0-1 背包问题：（**优先队列式求解**）

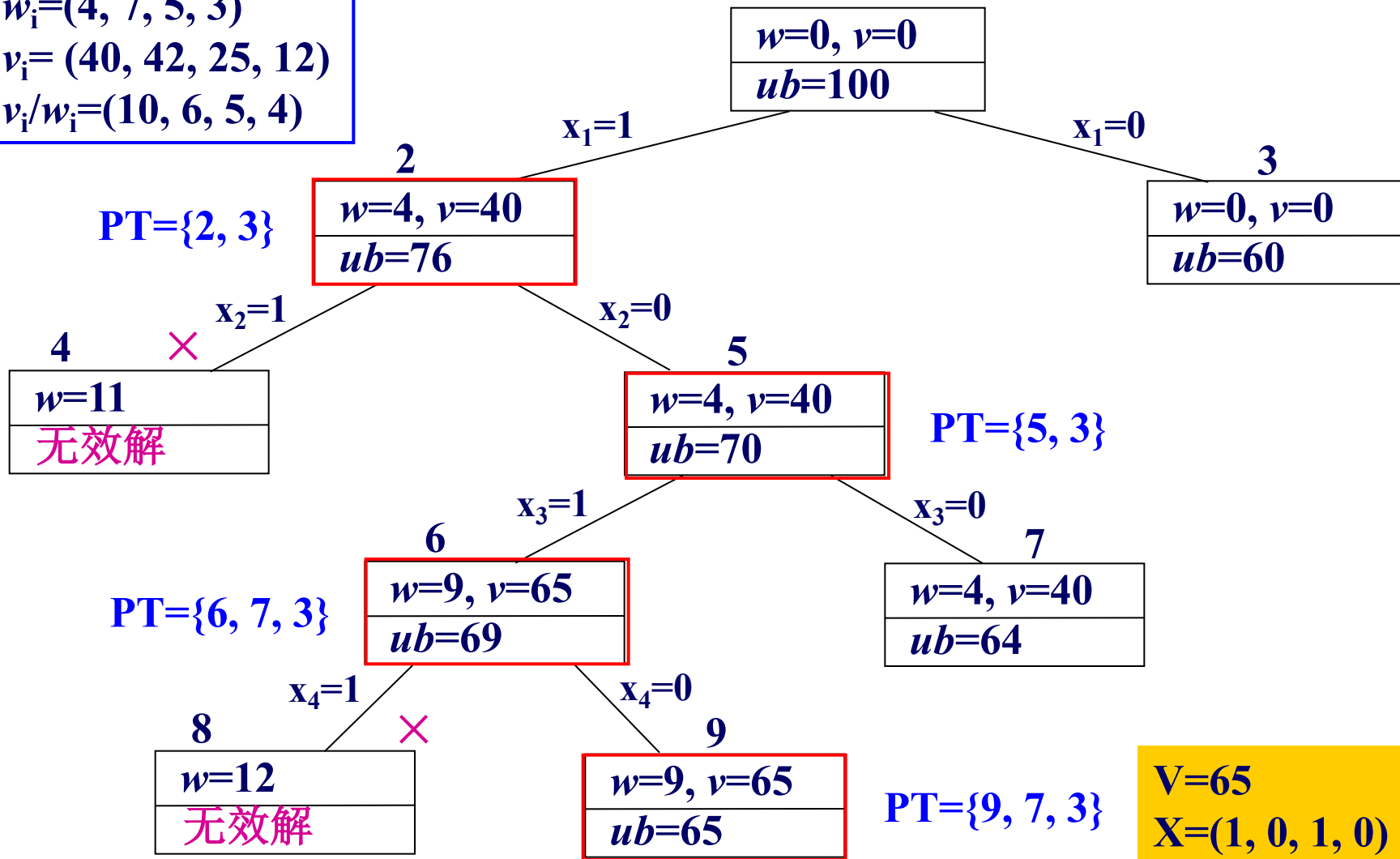
$n=4$, $C=10$, $w=\{4, 7, 5, 3\}$, $v=\{40, 42, 25, 12\}$

物品	重量(w)	价值(v)	价值/重量(v/w)
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

0-1 背包问题

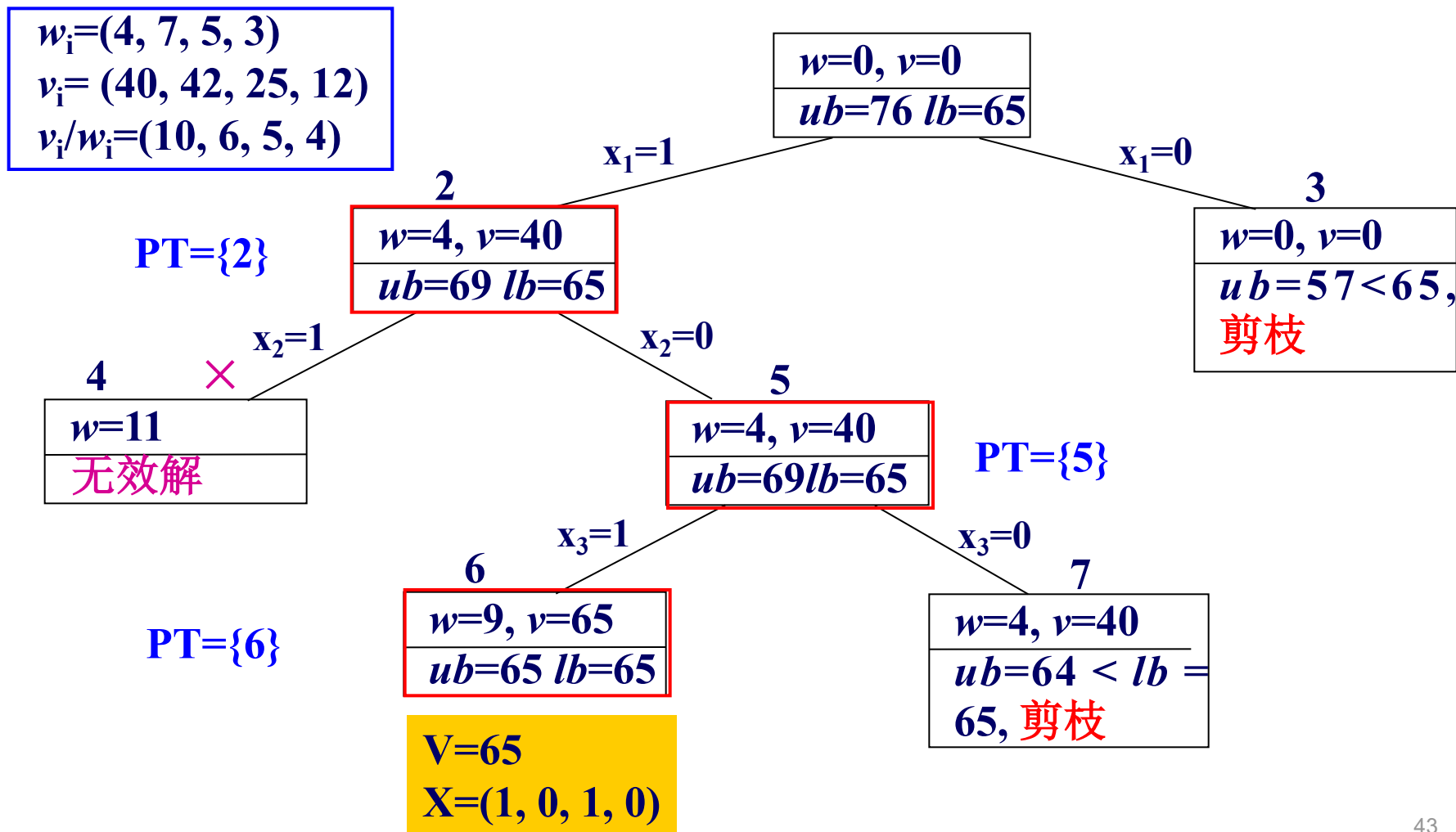
上界函数: $ub = v + (W - w) \times (v_{i+1} / w_{i+1})$

$w_i = (4, 7, 5, 3)$
 $v_i = (40, 42, 25, 12)$
 $v_i / w_i = (10, 6, 5, 4)$

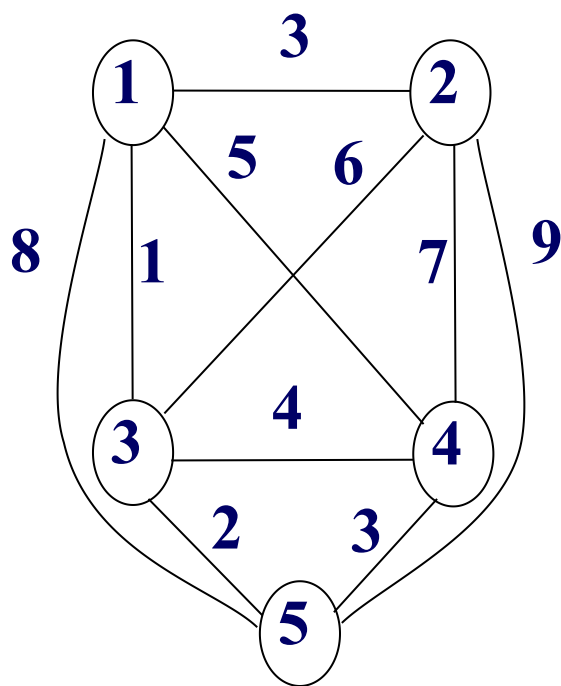


0-1 背包问题

上界函数：先删除大容量的物品，按照贪心方式给出，装不下了就拆分装满背包；
下界函数：按照贪心方式给出，装不下了不装了；



货郎担问题



(a) 一个无向图

$$C = \begin{bmatrix} \infty & 3 & 1 & 5 & 8 \\ 3 & \infty & 6 & 7 & 9 \\ 1 & 6 & \infty & 4 & 2 \\ 5 & 7 & 4 & \infty & 3 \\ 8 & 9 & 2 & 3 & \infty \end{bmatrix}$$

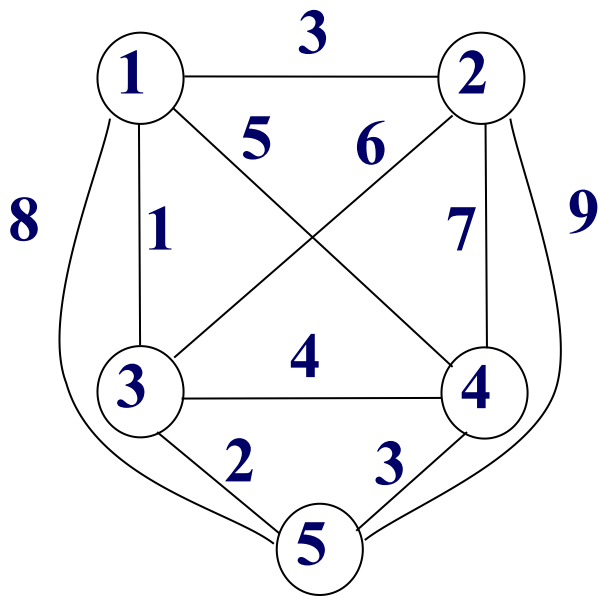
(b) 无向图的代价矩阵

货郎担问题

关于界的思考

► 问题上界:

贪心法可求得问题近似解 $1 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow 1$ ，其路径长度为 $1+2+3+7+3=16$ ，作为问题上界ub；



$$C = \begin{bmatrix} \infty & 3 & 1 & 5 & 8 \\ 3 & \infty & 6 & 7 & 9 \\ 1 & 6 & \infty & 4 & 2 \\ 5 & 7 & 4 & \infty & 3 \\ 8 & 9 & 2 & 3 & \infty \end{bmatrix}$$

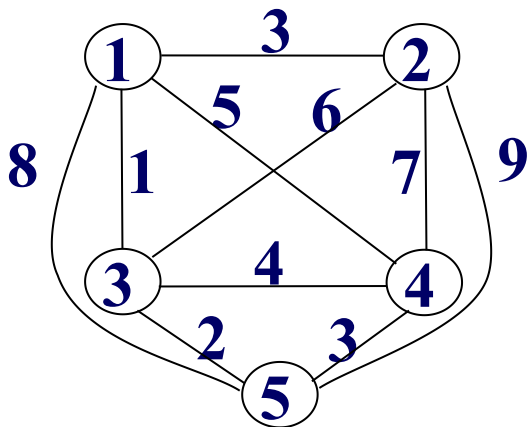
货郎担问题

关于界的思考

► 问题下界:

(1) 图邻接矩阵每行最小元素相加, $lb=1+3+1+3+2=10$;

(2) 每个顶点应该有出入两条不同的边, 将邻接矩阵中每行最小两个元素相加在除以2, 并向上取整, 可得到更好下界,
 $lb=((1+3)+(3+6)+(1+2)+(3+4)+(2+3))/2=14$;



$$C = \begin{bmatrix} \infty & 3 & 1 & 5 & 8 \\ 3 & \infty & 6 & 7 & 9 \\ 1 & 6 & \infty & 4 & 2 \\ 5 & 7 & 4 & \infty & 3 \\ 8 & 9 & 2 & 3 & \infty \end{bmatrix}$$

货郎担问题

► **部分解的下界**: U 表示已选结点的集合, 假设 U 中有 k 个元素 $\langle r_1, r_2, \dots, r_k \rangle$

$$lb = (2 \sum_{i=1}^{k-1} c[r_i][r_{i+1}] +$$

r_1 行不在路径上的最小元素 + r_k 行不在路径上的最小元素 +
 $\sum_{r_j \notin U} r_j$ 行最小的两个元素) / 2

假设 U 中只有3个结点, r_1, r_2, r_3 , 那么

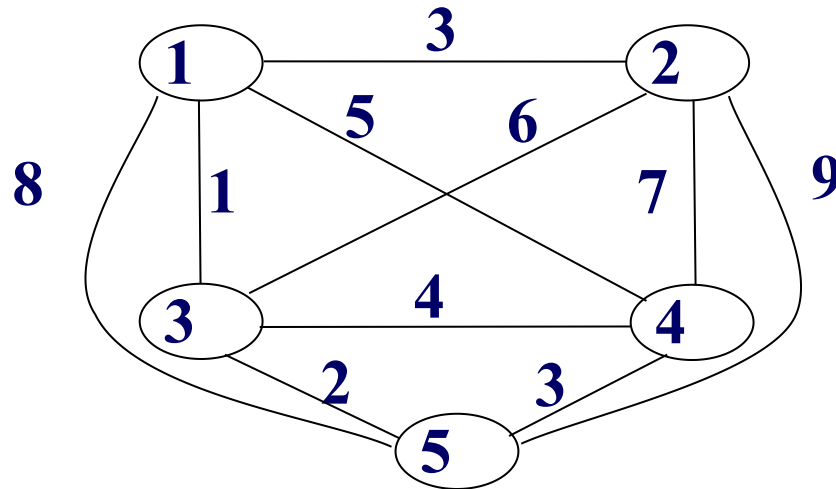
第一项相当于 $2c[r_1][r_2] + 2c[r_2][r_3]$; (共4项, 即4条边)

第二项相当于 r_1 行非 $c[r_1][r_2]$ 的最小值 + r_3 行非 $c[r_2][r_3]$ 的最小值;
(共2项, 即2条边)

前两项相一共6条边, 相当于是对 U 中的结点, 取最小的两个元素, 只是把最小的元素中的一部分换成了已选的路径

货郎担问题

► **部分解的下界**: U 表示已选结点的结合, 假设 U 中有 k 个元素 $\langle r_1, r_2, \dots, r_k \rangle$



例如: 设部分解为 $(1,4)$, 则该部分解的下界为:

$$lb = (2 * 5 +$$

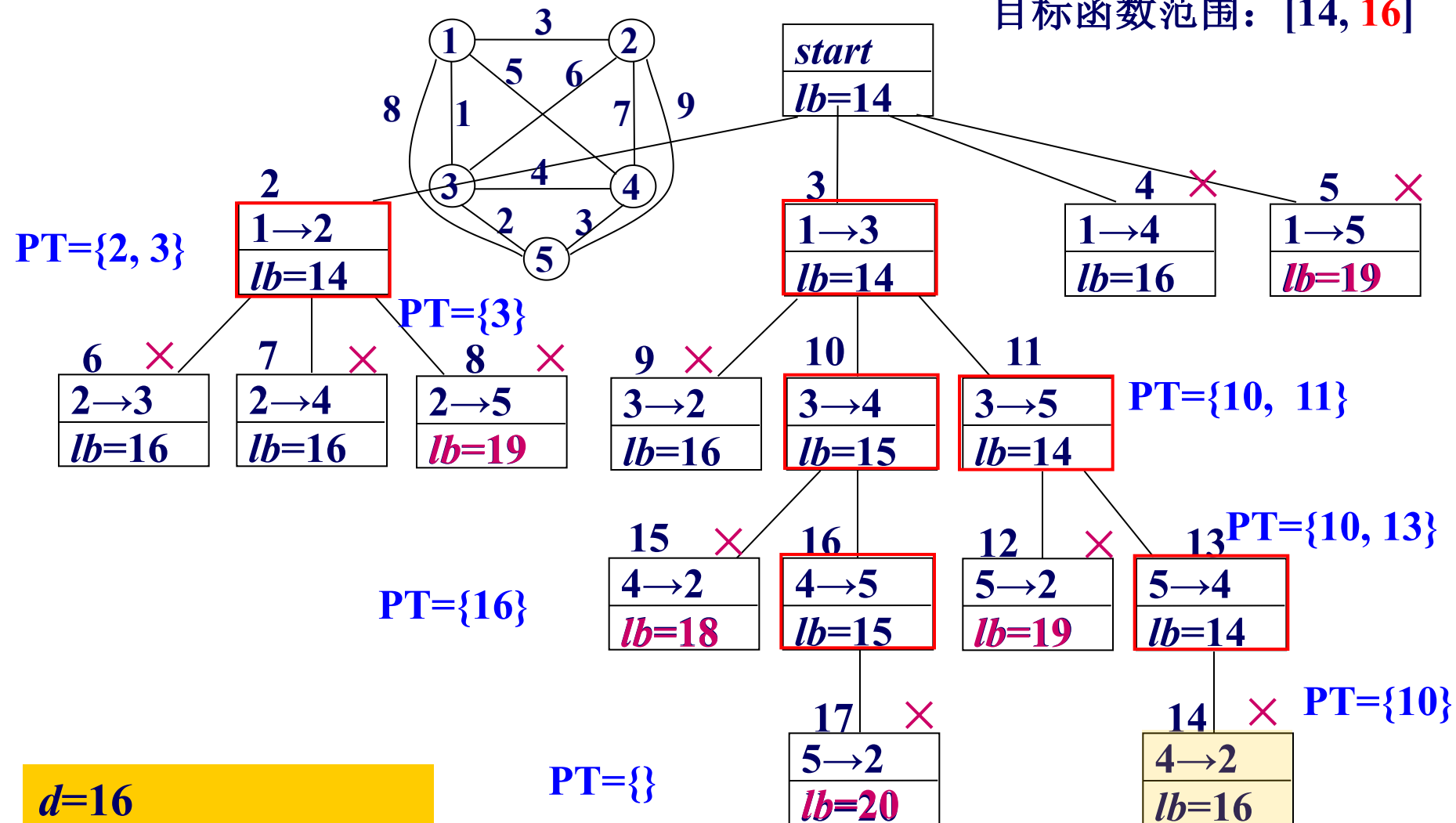
$$1 + 3 +$$

$$(3 + 6) + (1 + 2) + (2 + 3)) / 2 =$$

$$((1 + \textcolor{red}{5}) + (3 + 6) + (1 + 2) + (3 + \textcolor{red}{5}) + (2 + 3)) / 2 = 16$$

货郎担问题

目标函数范围: [14, 16]

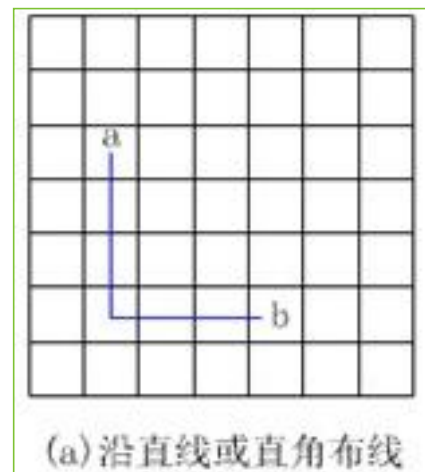
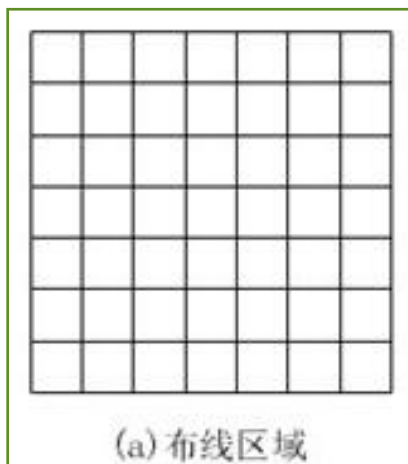


$d=16$

$1 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow 1$

布线问题

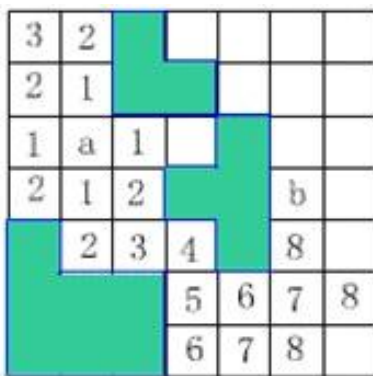
印刷电路板将布线区域划分成 $n \times m$ 个方格如图a所示。精确的电路布线问题要求确定连接方格a的中点到方格b的中点的最短布线方案。在布线时，电路只能**沿直线或直角布线**，如图b所示。为了避免线路相交，已布了线的方格做了**封锁标记**，其它线路**不允许穿过被封锁的方格**。



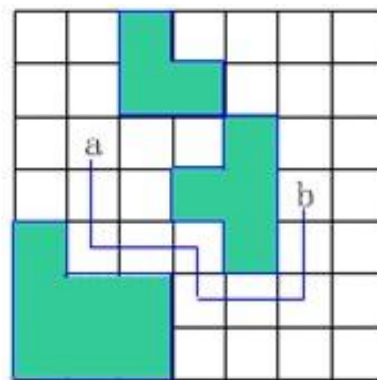
布线问题

算法思想： 队列式分支限界法从起始位置a开始将它作为第一个扩展结点。与该扩展结点相邻并且可达的方格成为可行结点被加入到活结点队列中，并且将这些方格标记为1，即**从起始方格a到这些方格的距离为1**。

接着，算法从活结点队列中取出队首结点作为下一个扩展结点，并将与当前扩展结点**相邻且未标记过的方格标记为2**，并存入活结点队列。这个过程**一直继续到算法搜索到目标方格b或活结点队列为空时为止**。即加入剪枝的广度优先搜索。



(a) 标记距离



(b) 最短布线路径

布线问题

```
Position [] offset = new Position [4];
```

```
offset[0] = new Position(0, 1);    // 右, 行不变, 列+1
```

```
offset[1] = new Position(1, 0);    // 下, 行+1, 列不变
```

```
offset[2] = new Position(0, -1);   // 左, 行不变, 列-1
```

```
offset[3] = new Position(-1, 0);   // 上, 行-1, 列不变
```

定义移动方向的
相对位移

```
for (int i = 0; i <= size + 1; i++)
```

```
{
```

```
    grid[0][i] = grid[size + 1][i] = 1; // 顶部和底部
```

```
    grid[i][0] = grid[i][size + 1] = 1; // 左翼和右翼
```

```
}
```

```
here = new Position(start.row, start.col)
```

设置边界的围墙

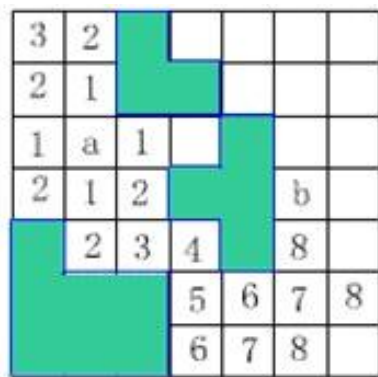
布线问题

```
while(true) {  
    for (int i = 0; i < numOfNbrs; i++){ // numOfNbrs=4, 相邻的结点数量  
        nbr.row = here.row + offset[i].row; //移动  
        nbr.col = here.col + offset[i].col;  
        if (grid[nbr.row][nbr.col] == 0) { // 该方格未标记, 标记了就不能继续移动了  
            grid[nbr.row][nbr.col] = grid[here.row][here.col] + 1; // 当前标记+1  
            if ((nbr.row == finish.row) && (nbr.col == finish.col)) break;  
            Q.put(new Position(nbr.row, nbr.col)); } // 加入活结点  
        }  
        // 是否到达目标位置 finish?  
        if ((nbr.row == finish.row) && (nbr.col == finish.col)) break; // 完成布线  
        if (Q.IsEmpty()) return false; // 活结点队列是否非空? 无解  
        here = Q.Poll(); // 取下一个扩展结点  
    }  
}
```

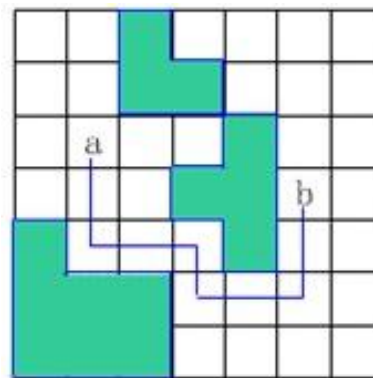

布线问题

如何构造最优解呢？

找到目标位置后，每次向标记位置标记值减1的方向回溯，直到找到初始结点即找到最短路径。



(a) 标记距离



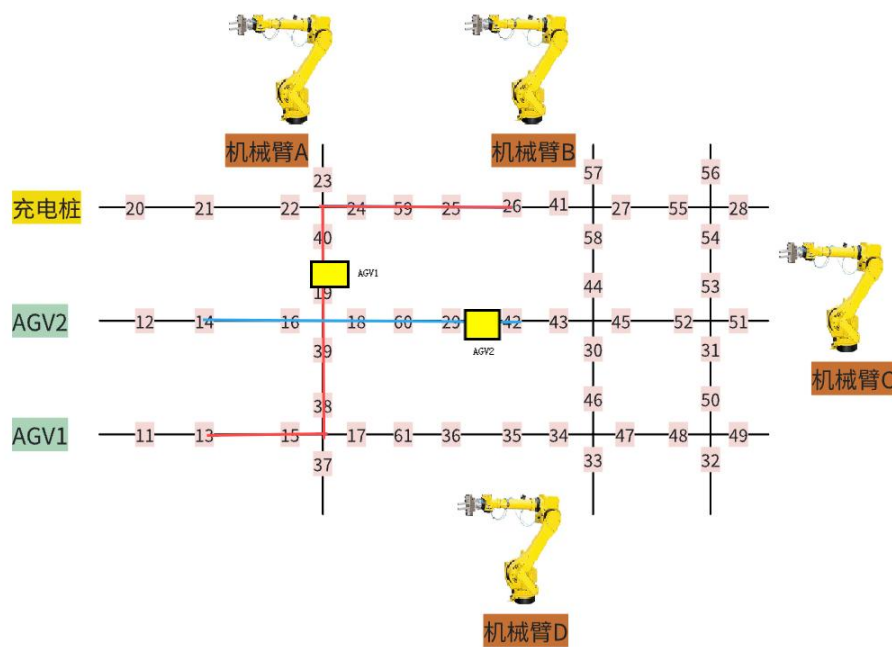
(b) 最短布线路径

布线问题—应用



布线问题还可以应用于对战，寻找最短路径

布线问题—拓展



如果给两个AGV规划好了路径，路径有交叉。假设算法可以实时获取AGV的位置，且能够实时控制AGV运动和停止。如何确保AGV走的过程中不会碰撞？怎么制定无碰撞策略？

最大团问题

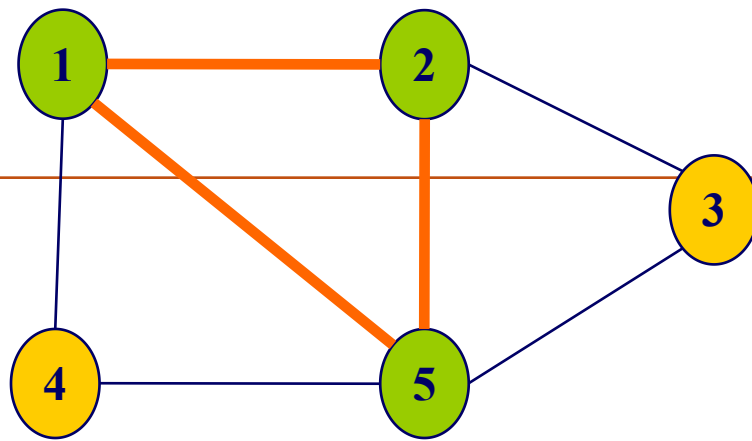
1. 问题描述

给定无向图 $G=(V, E)$ 。如果 $U \subseteq V$ ，且对任意 $u, v \in U$ 有 $(u, v) \in E$ ，则称 U 是 G 的完全子图。 G 的完全子图 U 是 G 的团当且仅当 U 不包含在 G 的更大的完全子图中。 G 的最大团是指 G 中所含顶点数最多的团。

最大团问题

1. 问题描述

下图G中，子集 $\{1, 2\}$ 是G的大小为2的完全子图。这个完全子图不是团，因为它被G的更大的完全子图 $\{1, 2, 5\}$ 包含。 $\{1, 2, 5\}$ 是G的最大团。 $\{1, 4, 5\}$ 和 $\{2, 3, 5\}$ 也是G的最大团。



最大团问题

2. 上界函数

用变量cliqueSize表示与该结点相应的团的顶点数；level表示结点在子集空间树中所处的层次，根结点 $level=1$ ；用 $cliqueSize+n-level+1$ 作为顶点数上界upperSize的值。

在此优先队列式分支限界法中，upperSize实际上也是优先队列中元素的优先级。算法总是从活结点优先队列中抽取具有最大upperSize值的元素作为下一个扩展元素。

分支限界法—最大团问题(PriorQueue)

数据结构:

Qnode {

Int upperSize; //最大团结点数上界

Int level; //所在层

Node parent; //父结点

Booleanan leftCh; //是否为左

} $\text{upperSize} = \text{cliqueSize} + n - \text{level} + 1$

最大团问题(PriorQueue)

3. 算法思想

子集树的根结点是初始扩展结点，对于这个特殊的扩展结点，其cliqueSize的值为0。

算法在扩展内部结点时，首先考察其左儿子结点。在左儿子结点处，将顶点 i 加入到当前团中，并检查该顶点与当前团中其他顶点之间是否有边相连。当顶点 i 与当前团中所有顶点之间都有边相连，则相应的左儿子结点是可行结点，将它加入到子集树中并插入活结点优先队列，并判断是否可以更新最优解，否则就不是可行结点。

最大团问题(PriorQueue)

3. 算法思想

接着继续考察当前扩展结点的右儿子结点。当 $\text{upperSize} > \text{bestn}$ 时，右子树中可能含有最优解，此时将右儿子结点加入到子集树中并插入到活结点优先队列中。

不断从优先队列中选取活结点，并按照上面的方式（先左子树，后右子树）扩展结点，直到满足终止条件。

最大团问题(PriorQueue)

4. 终止条件

算法的while循环的**终止条件**是遇到子集树中的一个**叶结点(即 $n+1$ 层结点)**成为当前扩展结点。

对于子集树中的叶结点，有 **$\text{upperSize} = \text{cliqueSize}$** 。此时活结点优先队列中剩余结点的upperSize值均不超过当前扩展结点的upperSize值，从而进一步搜索不可能得到更大的团，此时算法已找到一个最优解。

最大团问题(PriorQueue)

```
int bestn = 0;
Vector<Boolean> bestH = new Vector<Boolean>(); // 记录最优解
Node bestNode;
public void MaxCline(int[][] a, int n) {
1:  Qnode cNode = null; //当前扩展结点
2:  int i = 1; //结点的层
3:  int cn = 0; //当前结点数
4:  while(i != n + 1) //如果搜索到叶子结点层，且该结点的上界还是所有结点的
    上界中是最大的，那么从其它结点出发到达的叶子结点的最大装载量都不可能比
    这个值大{
5:      if(check(cNode, i)) { Ub = cliqueSize + n - level + 1
6:          if(cn + 1 > bestn) //更新bestn {
7:              bestn = cn + 1;
8:              //todo 一旦更新bestn记得清理heap中上界小于bestn的结点
9:              bestNode = new Qnode(cn + n - i + 1, i + 1, cNode, true);
10:             AddLiveNode(cn + n - i + 1, i + 1, cNode, true); }
```

最大团问题(PriorQueue)

```
11:      if(cn+n-i > bestn) //利用上界进行剪枝
12:          AddLiveNode(cn+n-i, i+1, cNode, false);
13:      if(heap.size() == 0) break; //取下一个结点
14:      cNode = heap.poll ();
15:      i = cNode.level;
16:      cn = cNode.upperSize - n + i - 1;
    }
17:  while(bestNode != null) { //保留最优解
        bestH.add(bestNode.leftChild);
        bestNode = bestNode.parent;
    }
}
```

$$Ub = cliqueSize + n - level + 1$$

批作业调度问题

1. 问题的描述

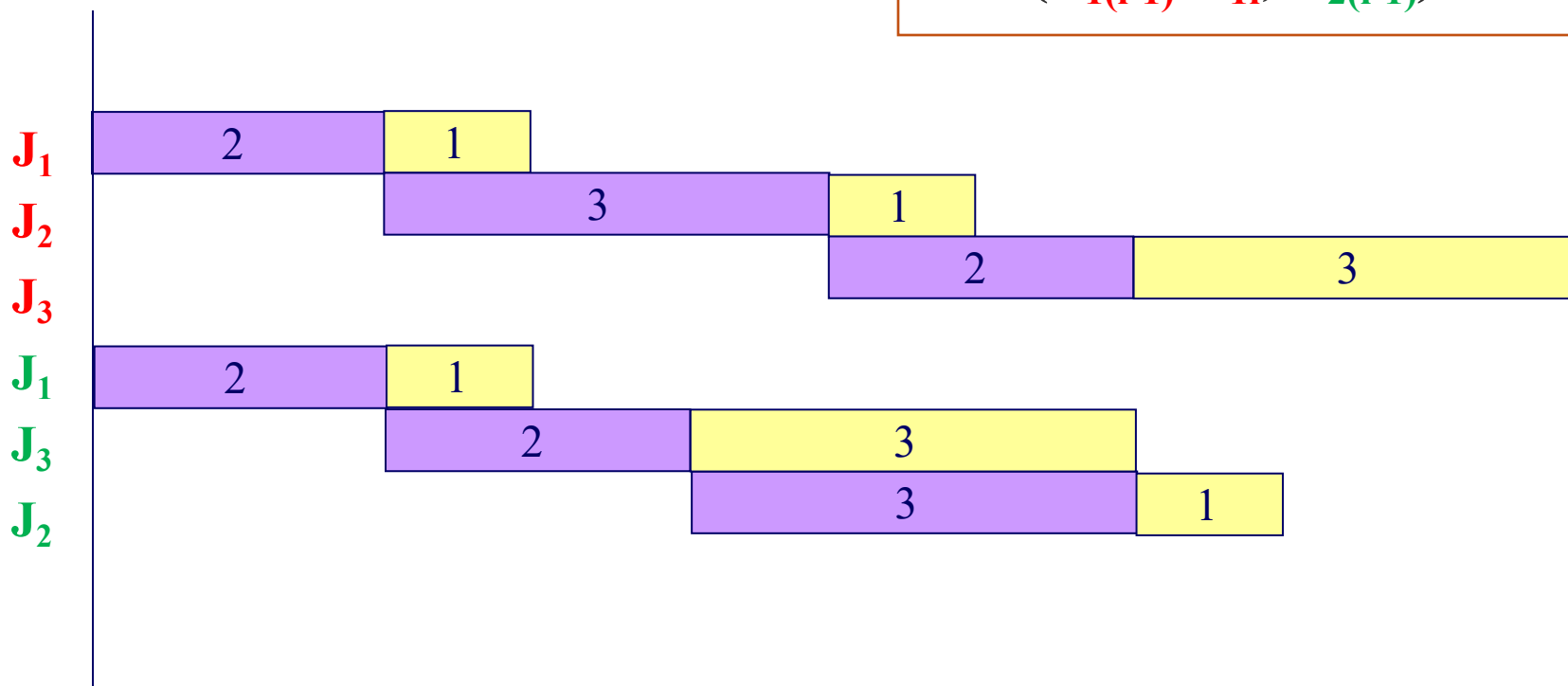
给定 n 个作业的集合 $J=\{J_1, J_2, \dots, J_n\}$ 。每一个作业 J_i 都有2项任务要分别在2台机器上完成。每一个作业必须先由机器1处理，然后再由机器2处理。作业 J_i 需要机器 j 的处理时间为 t_{ji} , $i = 1, 2, \dots, n$; $j = 1, 2$ 。

对于一个确定的作业调度，设是 F_{ji} 是作业 i 在机器 j 上完成处理的时间，则所有作业在机器2上完成处理的时间和为 $f = \sum_{i=1}^n F_{2i}$ 。求 f 的最小值？

批作业调度问题

t_{ji}	机器1	机器2
作业1	2	1
作业2	3	1
作业3	2	3

第*i*个工件在机器2上的开始时间为其在机器1上的完成时间与机器2上的第(*i*-1)个工件的完成时间取大即， $\max(F_{1(i-1)} + t_{1i}, F_{2(i-1)})$



批作业调度问题：下界

如果从结点E开始到叶结点的路上，机器1上没有空闲时间且工件在机器1上完成后立马能在机器2上加工，则剩余工件总完成时间为 S_1

$$\begin{aligned}
 S_1 = & (F_{1p_r} + t_{1p_{r+1}} + t_{2p_{r+1}}) + \\
 & (F_{1p_r} + t_{1p_{r+1}} + t_{1p_{r+2}} + t_{2p_{r+2}}) + \\
 & (F_{1p_r} + t_{1p_{r+1}} + t_{1p_{r+2}} + t_{1p_{r+3}} + t_{2p_{r+3}}) + \dots + \\
 & (F_{1p_r} + t_{1p_{r+1}} + t_{1p_{r+2}} + t_{1p_{r+3}} + \dots + t_{1p_{r+k}} + t_{2p_{r+k}}) \\
 = & \sum_{k=r+1}^n [F_{1p_r} + (n-k+1)t_{1p_k} + t_{2p_k}]
 \end{aligned}$$

p_r : 已安排的最后一个工件
 F_{1p_r} : p_r 在M1上的完成时间

K越小，前面的系数越大

批作业调度问题：下界

如果从结点E开始到叶结点的路上，机器2上没有空闲时间（前一个工件完成后立马能开始下一个工件），则剩余工件总完成时间为 S_2

$$\begin{aligned}
 S_2 &= \left\{ \max(F_{2p_r}, F_{1p_r} + \min_{i \notin M} t_{1i}) + t_{2p_{r+1}} \right\} + \\
 &\left\{ \max(F_{2p_r}, F_{1p_r} + \min_{i \notin M} t_{1i}) + t_{2p_{r+1}} + t_{2p_{r+2}} \right\} + \\
 &\left\{ \max(F_{2p_r}, F_{1p_r} + \min_{i \notin M} t_{1i}) + t_{2p_{r+1}} + t_{2p_{r+2}} + t_{2p_{r+3}} \right\} + \dots + \\
 &\left\{ \max(F_{2p_r}, F_{1p_r} + \min_{i \notin M} t_{1i}) + t_{2p_{r+1}} + t_{2p_{r+2}} + t_{2p_{r+3}} + \dots + t_{2p_n} \right\} \\
 &= \sum_{k=r+1}^n \left[\max(F_{2p_r}, F_{1p_r} + \min_{i \notin M} t_{1i}) + (n - k + 1)t_{2p_k} \right]
 \end{aligned}$$

M : 为已经安排的工件集合
 p_r : 已安排的最后一个工件
 F_{1pr} : p_r 在M1上的完成时间
 F_{2pr} : p_r 在M2上的完成时间

K越小，前面的系数越大

批作业调度问题

2. 下界函数

给定任意一个一个未加工工件的排序，其完成时间和大于 f :

$$f = \sum_{i \in M} F_{2i} + \max\{S_1, S_2\} = \sum_{i \in M} F_{2i} +$$
$$\max\left\{ \sum_{k=r+1}^n [F_{1p_r} + (n-k+1)t_{1p_k} + t_{2p_k}], \right.$$
$$\left. \sum_{k=r+1}^n [\max(F_{2p_r}, F_{1p_r} + \min_{i \notin M} t_{1i}) + (n-k+1)t_{2p_k}] \right\}$$

K越小，前面的系数越大

K越小，前面的系数越大

注意到如果选择 P_k ，使 t_{1p_k} 在 $k \geq r+1$ 时依非减序排列， S_1 则取得极小值。同理如果选择 P_k 使 t_{2p_k} 依非减序排列，则 S_2 取得极小值。

$$f \geq \sum_{i \in M} F_{2i} + \max\{\hat{S}_1, \hat{S}_2\}$$

这可以作为优先队列式分支限界法中的下界函数。

批作业调度问题结点设计

选择和扩展结点需要信息：

- 优先级信息 (lb) ，根据优先级信息确定每次扩展需要选取哪个活结点；
- 深度或者层数信息 level，只有知道了层的信息才知道要确定第几个作业；
- 当前作业调度 x，x 初值存储所有的作业编号，深度为 level 时，将 $x[\text{level}] \sim x[n]$ 的值依次交换到 $x[\text{level}]$ ，否则需要花费 $O(n)$ 时间计算可选作业；

批作业调度问题结点设计

算约束和界需要信息：

➤ 当前调度的完成时间和sf2（界函数中的 F_{2i} 求和）、以及两个机器当前的完成时间 f ，算界需要，不然需要递归到根结点，重复计算；

$$\sum_{i \in M} F_{2i} + \max\{S_1, S_2\} = \sum_{i \in M} F_{2i} +$$

$$\max\left\{\sum_{k=r+1}^n [F_{1p_r} + (n-k+1)t_{1p_k} + t_{2p_k}],\right.$$

$$\left.\sum_{k=r+1}^n [\max(F_{2p_r}, F_{1p_r} + \min_{i \notin M} t_{1i}) + (n-k+1)t_{2p_k}]\right\}$$

3. 算法描述

算法的while循环完成对排列树内部结点的有序扩展。在while循环体内算法依次从活结点**优先队列**中取出具有**最小lb值**（完成时间和的下界）的结点作为当前扩展结点cNode，并加以扩展。

批作业调度问题

3. 算法描述

首先考虑 $cNode.level=n$ 的情形，当前扩展结点 $cNode$ 是排列树中的 **叶结点**。
 $cNode.sf2$ 是相应于该叶结点的完成时间和。
输出 $cNode.x$ 对应的解。结束搜索过程。

批作业调度问题

3. 算法描述

当 $cNode.level < n$ 时，算法依次产生当前扩展结点 $cNode$ 的所有儿子结点。

对于当前扩展结点的每一个儿子结点 $node$ ，计算出其相应的完成时间和的下界 lb 。当 $lb < bestc$ 时，将该儿子结点插入到活结点优先队列中。而当 $lb \geq bestc$ 时，可将结点 $node$ 舍去。

批作业调度问题

3. 算法描述

cNode为当前搜索结点，每个结点为结构体，包含：**当前作业调度x**，完成时间和sf2，两个机器的完成时间f[]，下界lb，和安排工件数level

```
while (cNode.level <= n)
```

```
{
```

```
    if (cNode.level == n ) { // 叶结点，下界 最小
```

```
        if (cNode.sf2 < bestc) {
```

```
            bestc = cNode.sf2;
```

```
            for (int i = 0; i < n; i++)
```

```
                bestx[i] = cNode.x[i];
```

```
        }
```

```
        break; // 王老师的书上没有break，我觉得得加上
```

```
    }
```

有问题，应该直接更新当前最优值bestc和相应的最优解bestx

批作业调度问题

3. 算法描述

cNode为当前搜索结点，每个结点为结构体，包含：当前作业调度x，完成时间和sf2，两个机器的完成时间f[]，下界lb，和安排工件数level

else // 产生当前扩展结点的儿子结点

for (int i = cNode.level; i ≤ n; i++) { // 从所在的层s开始搜索

MyMath.swap(cNode.x, cNode.level, i); // 交换x的level,

int [] f= new int [3]; // f为两个机器的完成时间

int lb=bound(cNode, f); // 计算界值, 更新f[]

if (lb < bestc) {// 用界值剪枝

HeapNode node=new HeapNode(cNode, f, lb); // 新建结点会更新sf2, x, level

heap.put(node); }

MyMath.swap(cNode.x, cNode.level, i); // 为下一次交换准备

} // 完成结点扩展

// 取下一个结点

} // end while

当lb < bestc时，将儿子结点插入到活结点优先队列中

问题思考

对于极大化问题，如何给出一个好的上界？

考虑所有未装的工件以及自身的容量；

问题思考

对于极小化问题，如何给出一个好的下界？

考虑所有余下的边，每一行取两个小；
考虑所有剩余的工件，理想化的安排方案；

问题思考

如何判断界值设计的好坏？

随机产生案例，统计算法运行时间。

如何有多个上界或者下界，该怎么用？

可以用上界取 $\min$ 作为上界，用下界取 $\max$ 作为下界。

问题思考

回溯法不用太考虑界的设计，分支定界需要认真考虑界，是这样吗？

回溯法和分支定界法分别什么时候用呢？

总结

- 适应于求解组合搜索问题（含组合优化问题）
- 求解条件：满足多米诺性质
- 解的表示：子集树或者排列树
- 分支条件：约束条件或者界条件
- 分支策略：FIFO, PriorQueue
- 降低时间复杂性的主要途径：利用界函数对搜索树进行剪枝。难点在于界的设计。

讨论

- 问题一：分支限界法采用的宽度优先或函数优先的搜索方式，如果解空间太大，那么无论怎么剪枝都不能到达叶结点，这会导致什么问题？
- 答：解空间太大时导致的问题：很难搜索到叶子结点，导致很难找到问题的解。

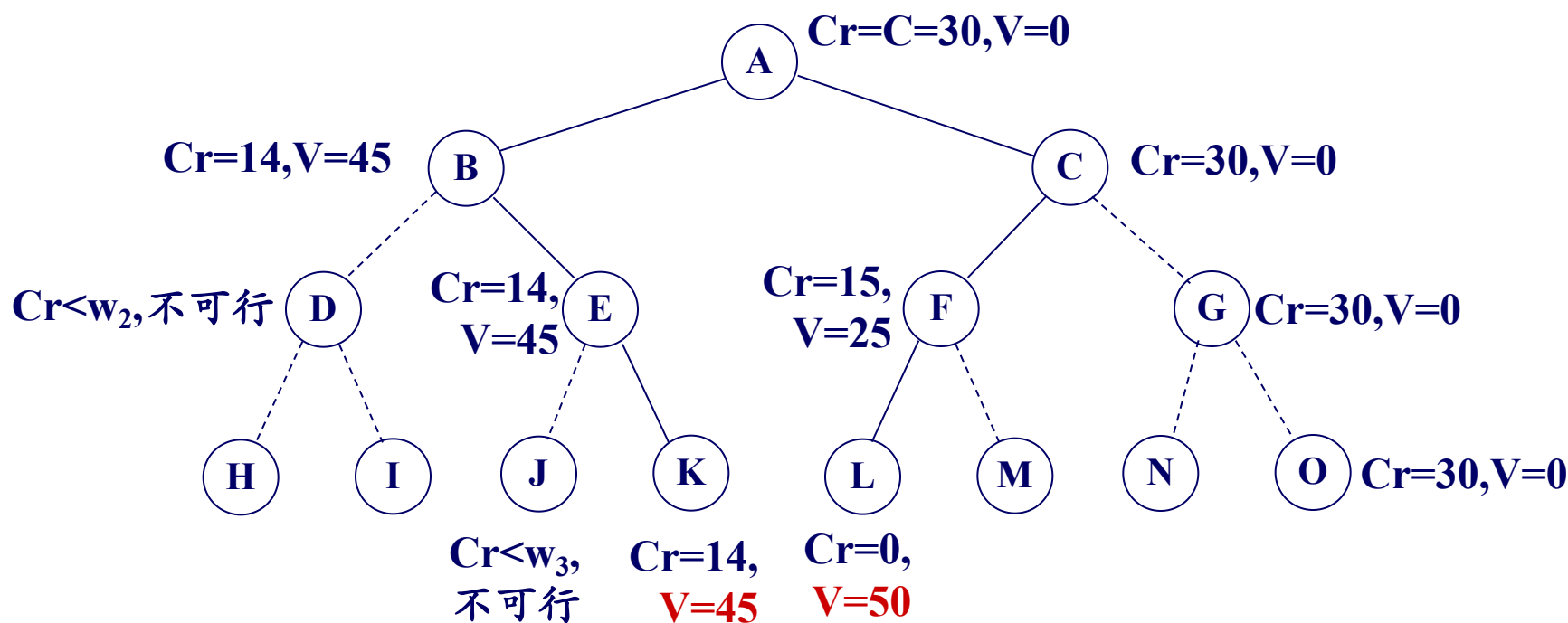
讨论

- 问题二：如何能够避免该问题？
- 答：可以采用启发式函数非admissible的A*的搜索方式；A*+搜索过程加搜索窗口的方式，限制每层结点数数量，确保能找到解的同时尽量确保解的质量；函数和深度优先结合的方式，在保证搜索结点的质量的同时，加快搜索进度；或者循环深度优先的方式，不断从根结点重新尝试新的搜索路径。

分支限界法基本思想

0-1 背包问题：（分析队列式与价值优先队列式过程）

$n=3$, $C=30$, $w=\{16, 15, 15\}$, $v=\{45, 25, 25\}$



A	B	C	E	F	K	L
---	---	---	---	---	---	---

A	B	C	E	K	F	L
---	---	---	---	---	---	---

分支限界法基本思想

0-1 背包问题：（分析队列式与价值优先队列式过程）

$n=3$, $C=30$, $w=\{16, 15, 15\}$, $v=\{45, 25, 25\}$

